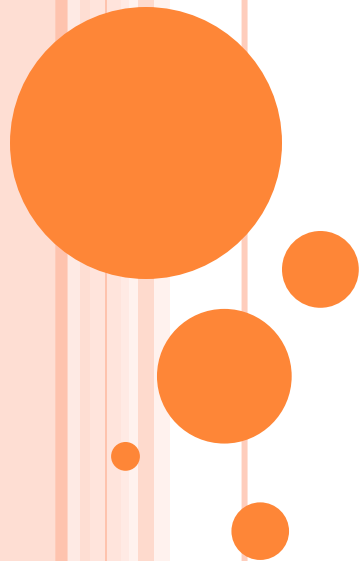


CORSO DI LAUREA IN INFORMATICA

CALCOLO SCIENTIFICO PROF. DAJANA CONTE

Errore di arrotondamento e fallimento del missile
Patriot



ESERCIZIO

Poni $a=0$. Poi aggiungi 0.625 ad a e fermati quando il valore di a diventa uguale a 5, stampando ad ogni passo il nuovo valore di a . Quanti passi occorrono?

MATLAB

```
a=0
while a~=5.0
    a=a+0.625
end
```

PYTHON

```
a=0
while a != 5.0:
    a=a+0.625
    print(a)
```

STRUTTURA ITERATIVA: CICLO WHILE

Le operazioni contenute nel ciclo *while* sono ripetute solo se la *condizione* di entrata è VERA. La condizione è costituita da un'espressione logica.

MATLAB

```
while (condizione)  
    istruzioni  
end
```

PYTHON

```
while (condizione):  
    istruzioni
```

ESPRESSIONI LOGICHE

Si ottengono applicando ad espressioni numeriche gli operatori di relazione:

MATLAB

- $\sim =$ Diverso
- $==$ Uguale
- $<$ Minore
- $<=$ Minore o Uguale
- $>$ Maggiore
- $>=$ Maggiore o Uguale

PYTHON

- $!=$
- $==$
- $<$
- $<=$
- $>$
- $>=$

Il risultato è: $\begin{cases} 0 & \text{se la relazione è falsa} \\ 1 & \text{se la relazione è vera} \end{cases}$

Per scrivere $\sim$ digitare 126 tenendo premuto Alt

LE ESPRESSIONI LOGICHE SI POSSONO COLLEGARE
USANDO GLI OPERATORI LOGICI:

MATLAB

&

AND

|

OR

~

NOT

PYTHON

and

or

not

L1	L2	L1 AND L2	L1 OR L2	NOT(L1)
1	1	1	1	0
1	0	0	1	0
0	1	0	1	1
0	0	0	0	1

ESEMPIO

Poni $a=0$. Poi aggiungi 0.1 ad a e fermati quando il valore di a diventa uguale ad 1, stampando ad ogni passo il nuovo valore di a . Quanti passi occorrono?

PYTHON

```
a=0
while a!=1.0 :
    a=a+0.1
    print(a)
```

MATLAB

```
a=0
while a~=1.0
    a=a+0.1
end
```

Il programma non si arresta mai!

Ma cosa è successo??

Convertire i numeri 0.625 e 0.1 in base 2.

0.625×2	1.25	1	b_{-1}
0.25×2	0.5	0	b_{-2}
0.5×2	1.0	1	b_{-3}
0.0×2	0.0	0	b_{-4}

$$(0.625)_{10} = (0 . \mathbf{101})_2$$

0.1×2	0.2	0	b_{-1}
0.2×2	0.4	0	b_{-2}
0.4×2	0.8	0	b_{-3}
0.8×2	1.6	1	b_{-4}
0.6×2	1.2	1	b_{-5}

0.2 ...

$$(0.1)_{10} = (0 . \mathbf{00011} 0011 0011 \dots)_2$$

Come mai il programma non si è arrestato?

$$(0.625)_{10} = (0 . \textcolor{red}{101})_2$$

Il numero 0.625 nella rappresentazione binaria ha un numero finito di cifre.

$$(0.1)_{10} = (0 . \textcolor{red}{00011} 0011 0011 \dots)_2$$

Il numero 0.1 nella rappresentazione binaria ha un numero infinito di cifre: è periodico illimitato.

Pertanto 0.1 non può essere rappresentato esattamente nella memoria del calcolatore, ma viene *approssimato*. Per questo a non diventa mai “esattamente” uguale ad 1.

Quale errore viene commesso nell'approssimazione?

Come modificare il ciclo while?

Per capirlo bene, dobbiamo comprendere come i dati vengono memorizzati in un calcolatore e le conseguenze di questa memorizzazione.



Per capire cosa accade scriviamo un ciclo for per eseguire la somma soltanto 10 volte.

CICLO FOR IN MATLAB

```
for cont = inizio: passo :fine  
    Istruzioni  
end
```

Quando *passo* è assente
si assume *passo*=1

CICLO FOR IN PYTHON

```
for cont in range(inizio,fine+1,passo):  
    Istruzioni
```



Per capire cosa accade scriviamo un ciclo for per eseguire la somma soltanto 10 volte:

MATLAB

```
a=0;  
for i=1:10  
    a=a+0.1  
end
```

PYTHON

```
a=0  
for i in range(1,11):  
    a=a+0.1  
print(a)
```

Eseguendo il programma in Matlab otteniamo:

```
>> a=1.0000
```

Ma se digitiamo:

```
>> format long e
```

Allora otteniamo:

```
>> a=9.9999999999999999e-001
```

Questo valore è “molto vicino ad 1”, ma non è esattamente uguale ad 1 !!!

Calcoliamo l'errore commesso:

```
>> err=abs(a-1)/abs(1)
```

Otteniamo $\text{err}=1.1102\text{e-}016$ (< della precisione di macchina eps)

Come fare per “obbligare” Matlab (Python) ad arrestare il ciclo while dopo 10 passi? Sostituiamo la condizione $a \sim 1$ ($a \neq 1$) con la condizione $\text{abs}(a-1) > \text{eps}$, in modo tale da considerare a diversa da 1 quando differisce da 1 per più della precisione di macchina eps (ovvero consideriamo $a=1$ quando differisce da 1 per meno della precisione di macchina eps).

MATLAB

```
a=0;  
while abs(a-1)>eps  
    a=a+0.1  
end
```

PYTHON

```
from sys import float_info  
eps=float_info.epsilon  
a=0  
while abs(a-1)>eps:  
    a=a+0.1  
    print(a)
```

In questo modo il ciclo while si arresta dopo 10 iterazioni.

ERRORI DOVUTI AD UN CATTIVO CALCOLO NUMERICO

<http://www.ima.umn.edu/~arnold/disasters/>

1



Fallimento del missile Patriot in Dharan, Arabia Saudita, il 25 Febbraio 1991: 28 morti. Attribuito ad uno scarso trattamento degli errori di arrotondamento. Un missile Patriot americano doveva intercettare uno Scud irakeno ma ha mancato lo scud per mezzo Km.

2



Esplosione immediatamente dopo la partenza del razzo Ariane 5 lanciato dall'Agenzia Spaziale Europea nella Guiana Francese, il 4 Giugno 1996. Conseguenza di un semplice overflow.

3



Affondamento della piattaforma Sleipner A per la produzione di petrolio e gas in Gandsfjorden near Stavanger, Norvegia, il 23 Agosto 1991. Causata da un'analisi inaccurata delle tecniche numeriche usate.

Una catastrofe dovuta ad errori nella rappresentazione dei numeri

25/2/1991 Dhara, Arabia Saudita Guerra del Golfo

Un missile Patriot americano deve intercettare uno Scud irakeno

L'orologio interno dei Patriot scandisce il tempo in decimi di secondo.

$1/10$ nella rappresentazione binaria è periodico

$$\frac{1}{10} = 0,000110011001100110011001100...$$

Il registro interno del patriot ha a disposizione 24 bit per rappresentare con i dati

$$\frac{1}{10} = 0,00011001100110011001100$$

Pertanto l'errore commesso è

$$(0,0000000000000000000000001100...)_{2} \approx (0,000000095)_{10}$$

Il sistema di intercettazione del missile era in funzione da 100 ore, aveva pertanto accumulato un errore di

$$9.5 \times 10^{-8} \times 10 \times 60 \times 60 \times 100 = 0,34 \text{ secondi}$$

Uno scud viaggia a 1.676 metri al secondo: il patriot ha fallito lo scud per mezzo Km.

Sono morti 28 soldati e 100 civili.